

Referencia de API Futcamedic

Documentación Completa de la API REST

Equipo de Ingeniería
Futcamedic

5 de mayo de 2026

Referencia de API Futcamedic

URL Base: <http://localhost:3000/api> (desarrollo) o <https://your-render-app.onrender.com/api> (producción)

Todos los endpoints requieren autenticación vía JWT de Supabase en el encabezado Authorization:

Authorization: Bearer <supabase_jwt_token>

El multi-tenancia se impone vía `school_id` del perfil del usuario autenticado.

Endpoints de Autenticación

POST /auth/register

Registra una nueva escuela con usuario admin.

Cuerpo de la Solicitud:

```
{
  "schoolName": "Academia de Fútbol XYZ",
  "fullName": "Juan Pérez",
  "email": "juan@xyz.com",
  "password": "SecurePass123!"
}
```

Respuesta:

```
{
  "message": "Escuela y admin creados exitosamente",
  "user": { "id": "uuid", "email": "juan@xyz.com" },
  "school": { "id": "uuid", "name": "Academia de Fútbol XYZ" }
}
```

POST /auth/invite

Invita un usuario (profesor, admin, padre). Requiere rol admin.

Cuerpo de la Solicitud:

```
{
  "email": "profesor@xyz.com",
  "fullName": "Carlos Gómez",
  "role": "profesor",
  "phone": "555-1234"
}
```

Respuesta:

```
{
  "message": "Usuario invitado exitosamente",
}
```

```
  "user": { "id": "uuid", "email": "profesor@xyz.com", "role": "profesor" }
}
```

Endpoints de Alumnos

GET /students

Lista alumnos. Admin ve todos; padre ve solo sus hijos.

Parámetros de Consulta: - category_id (opcional): Filtrar por categoría - status (opcional): Filtrar por estado (activo, beca, pendiente, inactivo)

Respuesta:

```
[
  {
    "id": "uuid",
    "full_name": "Juanito Pérez",
    "birth_date": "2012-05-15",
    "status": "activo",
    "uniform_delivered": false,
    "category": { "id": "uuid", "name": "U-12", "birth_year": 2012 },
    "parent": { "id": "uuid", "full_name": "María Pérez", "email": "maria@email.com" }
  }
]
```

POST /students

Crea un nuevo alumno. Requiere rol admin.

Cuerpo de la Solicitud:

```
{
  "fullName": "Juanito Pérez",
  "birthDate": "2012-05-15",
  "categoryId": "uuid",
  "parentEmail": "maria@email.com",
  "parentName": "María Pérez",
  "parentPhone": "555-5678",
  "medicalNotes": "No alergias"
}
```

GET /students/:id

Obtiene detalles del alumno con historial de asistencias y pagos.

PUT /students/:id

Actualiza información del alumno.

Cuerpo de la Solicitud:

```
{
  "fullName": "Juanito Pérez",
  "birthDate": "2012-05-15",
  "categoryId": "uuid",
  "parentId": "uuid",
  "medicalNotes": "Notas actualizadas"
}
```

PATCH /students/:id/status

Cambia estado del alumno.

Cuerpo de la Solicitud:

```
{
  "status": "beca"
}
```

Estados válidos: activo, beca, pendiente, inactivo

PATCH /students/:id/uniform

Alterna estado de entrega de uniforme.

Respuesta:

```
{
  "message": "Estado de uniforme actualizado",
  "uniform_delivered": true
}
```

DELETE /students/:id

Elimina alumno (permanente). Crea registro de auditoría en deleted_students.

GET /students/deleted

Lista alumnos eliminados (solo admin).

Endpoints de Asistencias

GET /attendances/category/:categoryId

Obtiene asistencias para una categoría, filtrado por fecha.

Parámetros de Consulta: - date (opcional): Filtrar por fecha (AAAA-MM-DD)

GET /attendances/student/:studentId

Obtiene historial de asistencias de un alumno.

POST /attendances

Guarda asistencias en bloque (pase de lista). Usa upsert para prevenir duplicados.

Cuerpo de la Solicitud:

```
{
  "trainingId": "uuid",
  "date": "2026-05-05",
  "type": "entrenamiento",
  "records": [
    { "studentId": "uuid", "present": true },
    { "studentId": "uuid", "present": false }
  ]
}
```

PATCH /attendances/trainings/:id/complete

Marca sesión de entrenamiento como completada.

Respuesta:

```
{
  "message": "Entrenamiento marcado como completado",
  "is_completed": true
}
```

Endpoints de Pagos

GET /payments

Lista pagos.

Parámetros de Consulta: - type (opcional): mensualidad o pago_profesor - month (opcional): Filtrar por mes (formato AAAA-MM, ej., "2026-05")

GET /payments/student/:studentId

Obtiene historial de pagos de un alumno específico.

GET /payments/pending

Obtiene alumnos sin pago en el mes actual.

POST /payments/students

Registra pago para alumno(s). Soporta pagos individuales o en bloque.

Cuerpo de la Solicitud (Individual):

```
{
  "studentId": "uuid",
  "amount": 500.00,
  "paymentDate": "2026-05-05",
  "description": "Mensualidad Mayo 2026"
}
```

Cuerpo de la Solicitud (En bloque):

```
{
  "studentIds": ["uuid1", "uuid2", "uuid3"],
  "amount": 1500.00,
  "paymentDate": "2026-05-05",
  "description": "Mensualidad Mayo 2026 (Grupal)"
}
```

Nota: Para pagos en bloque, el monto se divide equitativamente entre los alumnos y se registra en payment_students.

POST /payments/teachers

Registra pago a profesor (nómina).

Cuerpo de la Solicitud:

```
{
  "teacherId": "uuid",
  "amount": 2000.00,
  "paymentDate": "2026-05-05",
  "description": "Pago quincenal Mayo"
}
```

Endpoints de Eventos

GET /events

Lista eventos. Los profesores ven solo eventos para sus categorías.

Parámetros de Consulta: - category_id (opcional): Filtrar por categoría - month (opcional): Filtrar por mes (AAAA-MM)

GET /events/trainings

Obtiene sesiones de entrenamiento de hoy. Los profesores ven solo sus categorías.

Parámetros de Consulta: - date (opcional): Filtrar por fecha (AAAA-MM-DD), por defecto hoy

POST /events

Crea un nuevo evento con recurrencia opcional.

Cuerpo de la Solicitud (Evento Único):

```
{
  "categoryId": "uuid",
  "venueId": "uuid",
  "date": "2026-05-10",
  "startTime": "16:00",
  "type": "entrenamiento",
  "description": "Entrenamiento técnico"
}
```

Cuerpo de la Solicitud (Evento Recurrente - Semanal):

```
{
  "categoryId": "uuid",
  "venueId": "uuid",
  "date": "2026-05-10",
  "startTime": "16:00",
  "type": "entrenamiento",
  "description": "Entrenamiento semanal",
  "isRecurring": true,
  "recurringWeeks": 12,
  "recurrenceRule": {
    "frequency": "weekly",
    "days": ["Monday", "Wednesday", "Friday"]
  }
}
```

Cuerpo de la Solicitud (Evento Recurrente - Quincenal):

```
{
  "categoryId": "uuid",
  "venueId": "uuid",
  "date": "2026-05-10",
  "startTime": "16:00",
  "type": "entrenamiento",
  "isRecurring": true,
  "recurringWeeks": 8,
  "recurrenceRule": {
    "frequency": "biweekly",
    "day": "Saturday"
  }
}
```

GET /events/:id/trainings

Obtiene todas las sesiones de entrenamiento generadas de un evento recurrente.

POST /events/cancel

Cancela sesión(es) de entrenamiento.

Cuerpo de la Solicitud (Cancelar entrenamiento individual):

```
{
  "trainingId": "uuid",
  "notifyParents": true
}
```

Cuerpo de la Solicitud (Cancelar evento completo/recurrencia):

```
{
  "eventId": "uuid",
  "cancelAll": true,
  "notifyParents": true
}
```

DELETE /events/:id

Elimina un evento y todas sus sesiones de entrenamiento.

Endpoints de Categorías

GET /categories

Lista todas las categorías con profesor asignado.

GET /categories/mine

Obtiene categorías asignadas al profesor actual.

POST /categories

Crea una nueva categoría.

Cuerpo de la Solicitud:

```
{
  "birthYear": 2012,
  "name": "U-12"
}
```

PATCH /categories/:id

Actualiza categoría.

Cuerpo de la Solicitud:

```
{
  "name": "U-12 Elite",
  "birthYear": 2012
}
```

POST /categories/:id/assign

Asigna un profesor a una categoría.

Cuerpo de la Solicitud:

```
{
  "teacherId": "uuid"
}
```

Endpoints de Sedes

GET /venues

Lista todas las sedes.

POST /venues

Crea una nueva sede.

Cuerpo de la Solicitud:

```
{
  "name": "Campo Principal",
  "address": "Av. Reforma 123, Ciudad",
  "notes": "Césped artificial, vestidores disponibles"
}
```

PATCH /venues/:id

Actualiza información de la sede.

DELETE /venues/:id

Elimina una sede.

Endpoints de Usuarios

GET /users

Lista usuarios filtrados por rol.

Parámetros de Consulta: - role (opcional): Filtrar por rol (super_admin, admin, profesor, padre, alumno)

GET /users/teachers/:id

Obtiene detalles del profesor con categorías asignadas y email de Auth.

PUT /users/:id

Actualiza perfil de usuario.

Cuerpo de la Solicitud:

```
{
  "fullName": "Carlos Gómez",
  "phone": "555-1234",
  "address": "Calle 123",
  "emergencyContactName": "Ana Gómez",
  "emergencyContactPhone": "555-5678",
  "medicalNotes": "Ninguna"
}
```

PATCH /users/me/password

Cambia contraseña del usuario actual.

Cuerpo de la Solicitud:

```
{
  "currentPassword": "OldPass123!",
  "newPassword": "NewPass456!"
}
```

Endpoints de Permisos de Profesores

GET /permissions/mine

Obtiene permisos del profesor actual.

GET /permissions/:teacherId

Obtiene permisos de un profesor específico (solo admin).

PUT /permissions/:teacherId

Actualiza permisos de profesor (solo admin).

Cuerpo de la Solicitud:

```
{
  "can_manage_students": true,
  "can_manage_events": true,
  "can_view_finances": false,
  "can_manage_payments": false,
  "can_take_attendance": true,
  "can_manage_categories": false
}
```

Endpoints de Escuelas

PUT /schools/:id

Actualiza información de la escuela (solo admin).

Cuerpo de la Solicitud:

```
{
  "name": "Nueva Academia XYZ",
  "logoUrl": "https://supabase.storage.com/avatars/school-logo.png"
}
```

Respuestas de Error

Todos los endpoints devuelven errores en este formato:

```
{
  "error": "Mensaje de error describiendo qué salió mal"
}
```

Códigos de Estado HTTP Comunes: - 200 - Éxito - 201 - Creado - 400 - Solicitud Incorrecta (error de validación) - 401 - No Autorizado (token faltante/inválido) - 403 - Prohibido (rol/permisos insuficientes) - 404 - No Encontrado - 409 - Conflicto (registro duplicado) - 500 - Error Interno del Servidor

Pila de Middleware

Todas las solicitudes de API pasan por esta cadena de middleware:

1. **requireAuth** (api/middlewares/auth.middleware.ts)
 - Valida JWT de Supabase del encabezado Authorization: Bearer <token>
 - Configura req.user = { id, email, ...decodedJWT }
2. **requireTenant** (api/middlewares/tenant.middleware.ts)
 - Busca en tabla users usando req.user.id
 - Configura req.tenant = { school_id, role, user_id }
 - Rechaza si usuario no se encuentra en tabla users
3. **requireRole(...roles)** (aplicado por ruta)
 - Verifica si req.tenant.role está en la lista de roles permitidos
 - Rechaza con 403 si rol no autorizado

Limitación de Tasa

Actualmente no hay limitación de tasa implementada. Considerar agregar para producción: - Supabase Auth tiene limitación de tasa integrada para endpoints de auth - Considerar `express-rate-limit` para endpoints de API en producción